

UNIVERSITE DE CORSE
Master Informatique
parcours DFS et DE
2^{ème} année
2025-2026

CH4 – Des embeddings aux BD vectorielles : principes et premières explorations

Evelyne VITTORI
vittori_e@univ-corse.fr



Partie I - Représenter le monde : des symboles aux vecteurs



CH1 – Représenter
le monde en IA : un
enjeu central



CH2 –
Représentations en
IA symbolique : des
origines aux
ontologies



CH3 – Représenter
par des vecteurs :
fondements des
embeddings



CH4 – Indexation et
BD Vectorielles



Objectifs du chapitre

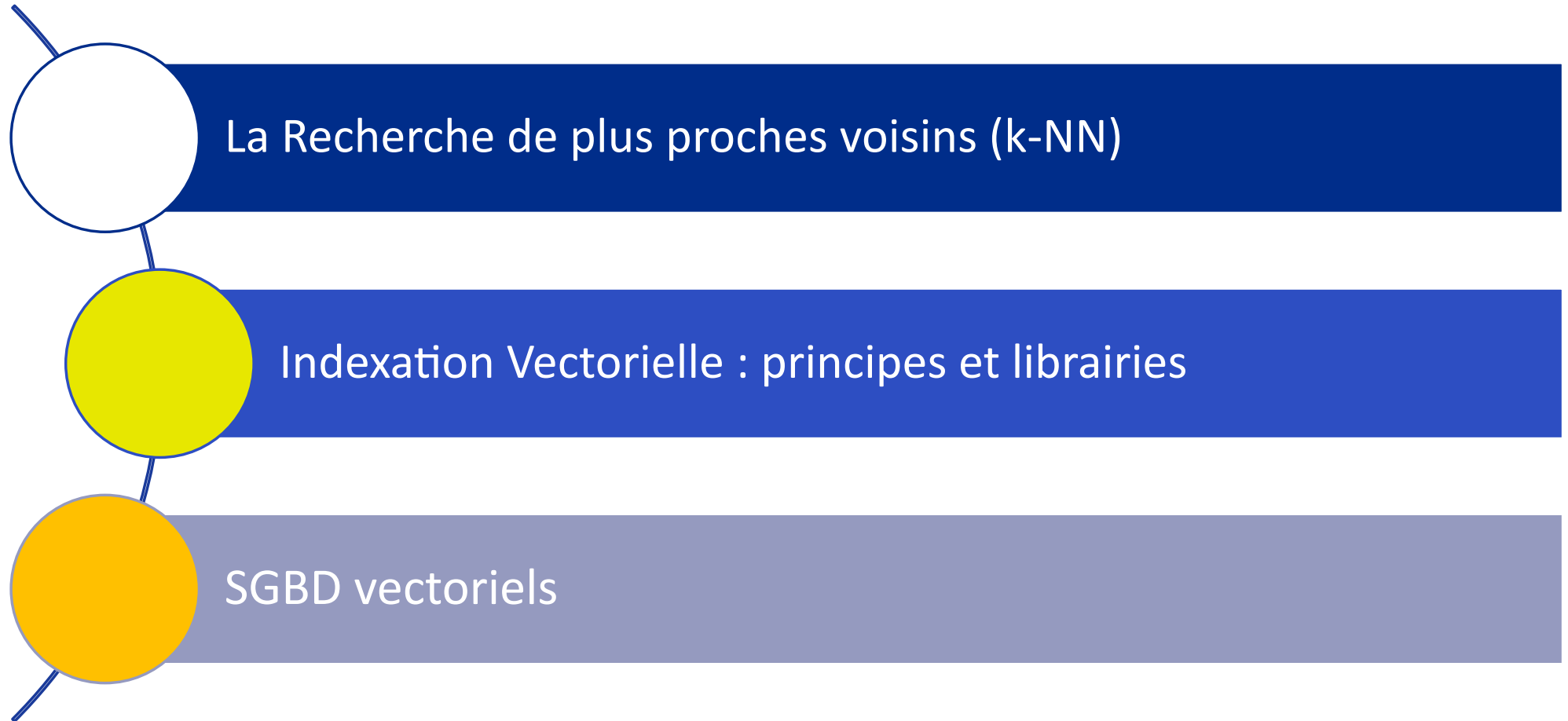
- Comprendre comment les vecteurs embeddings peuvent être **indexés et recherchés** efficacement (FAISS)
- Découvrir les **bases vectorielles simples** (Chroma)
- Poser les fondations de la **recherche sémantique** qui servira ensuite dans les systèmes RAG (partie 2)



Activités guidées

- Recherche k-NN
- Indexation ANN avec FAISS
- BD Vectorielle avec ChromaDB

CH4 – Indexation et BD vectorielles : principes et premières explorations



Pourquoi des bases vectorielles ?

- Les embeddings permettent de mesurer la similarité sémantique (ex cosinus)

Mais :

Problème 1 : **Scalabilité**

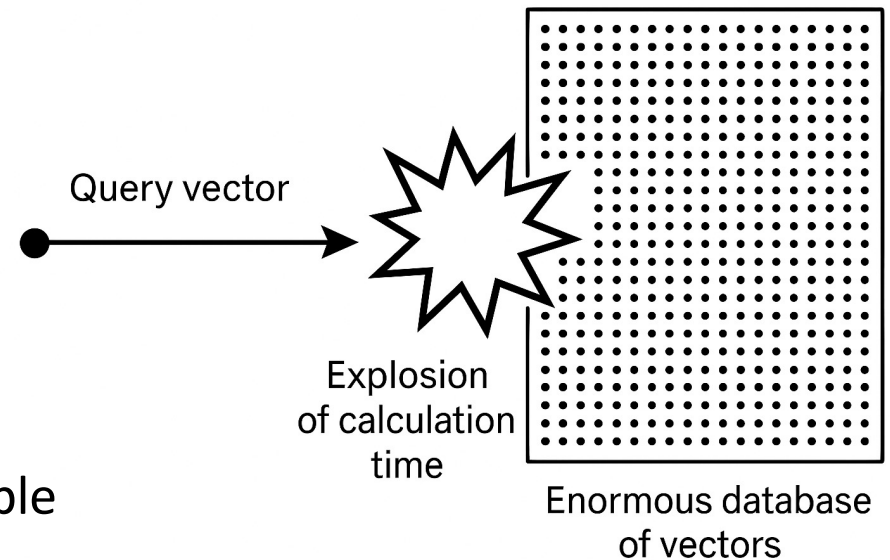
→ comparer un vecteur à millions d'autres devient trop lent

Problème 2 : **Efficacité mémoire**

→ stockage de vecteurs haute dimension

Problème 3 : **Recherche avancée**

→ besoin de fonctions plus riches qu'un simple "plus proche voisin" (filtrage, métadonnées, mises à jour temps réel)



La solution : Indexation & SGBD vectoriels



Outils d'indexation vectorielle



Accélérer la recherche de similarité



SGBD vectoriels



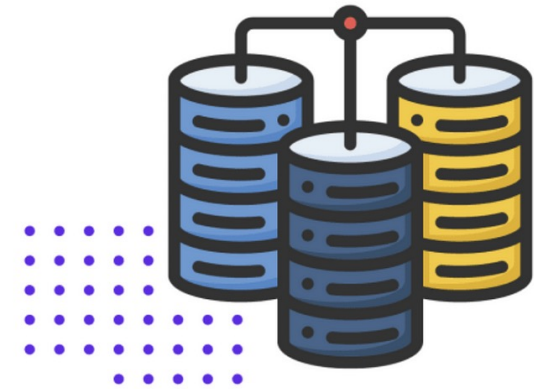
Chroma



Pinecone



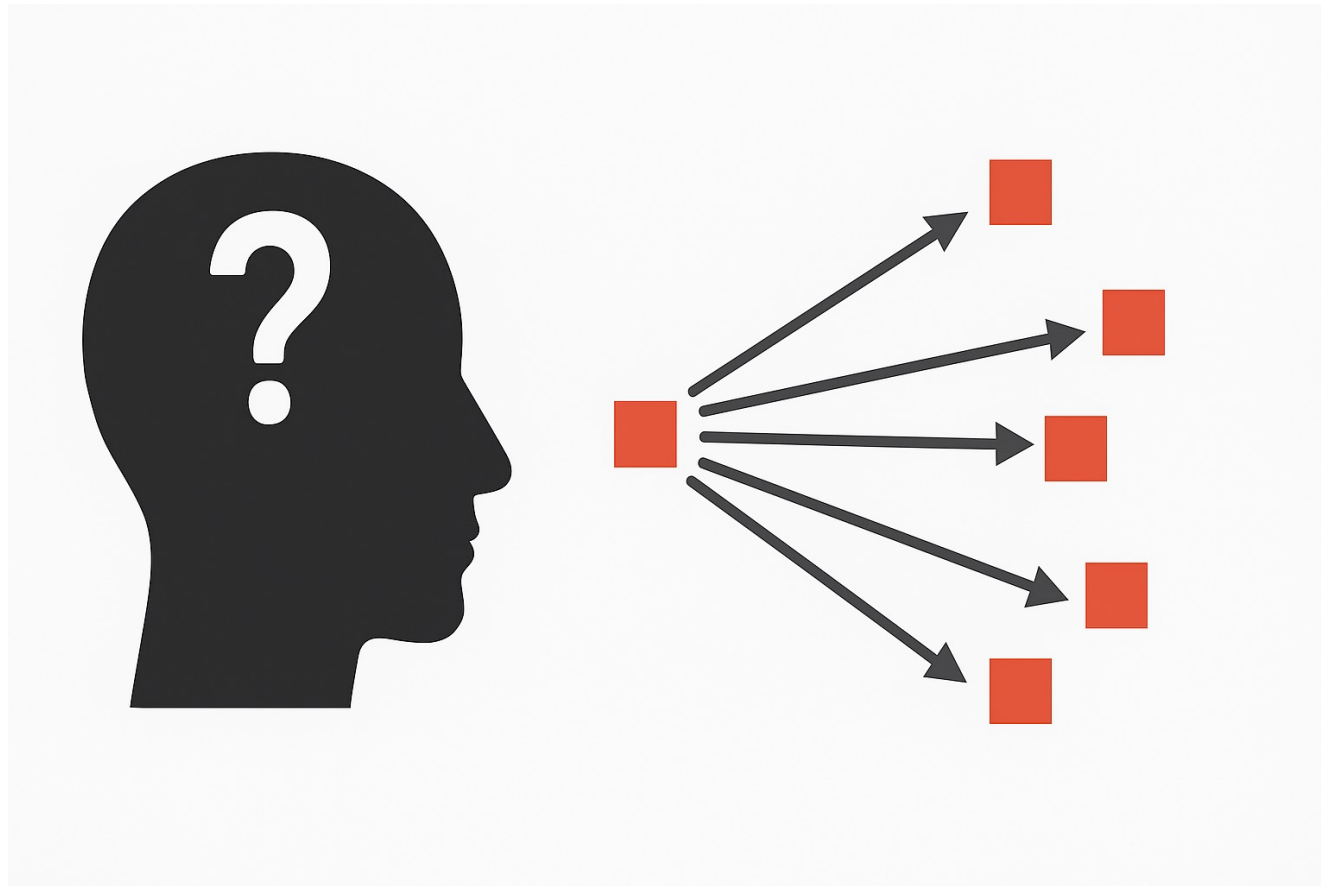
Aller plus loin : stockage persistant,
gestion des métadonnées, API de requête,
scalabilité



Fondations des systèmes **RAG** et moteurs de recherche
sémantiques modernes

1 – La recherche de plus proches voisins

k-NN : k-Nearest Neighbors

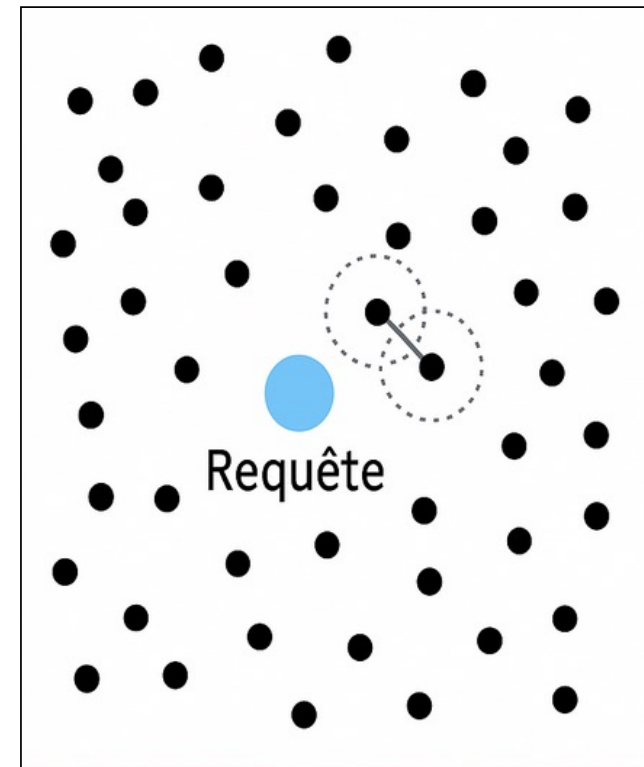


k-NN avec embeddings ?

- Objectif = Trouver les k vecteurs les plus similaires à une requête (*vecteur q*) dans un corpus de vecteurs

$$X = \{x_1, x_2, \dots, x_N\}$$

- Exemple : *“Quel document est le plus proche de “roi”?”*
- Méthode : calculer une **distance** ou une **similarité** entre vecteurs
- Distances classiques :
 - Euclidienne
 - Cosinus (souvent préférée pour les embeddings texte)



Distance euclidienne vs similarité cosinus (rappel)

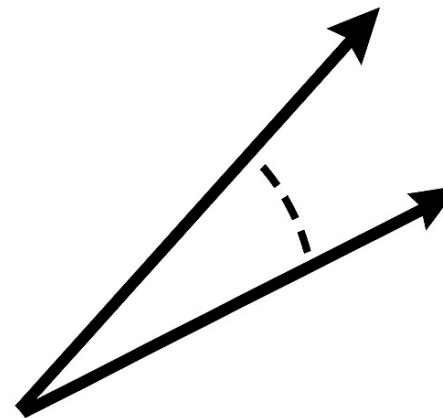
- **Distance euclidienne** : mesure “géométrique”
 - compare longueur + position absolue
 - proche si les vecteurs sont vraiment proches dans l’espace



Exemple : deux documents sur le même thème mais de tailles différentes

- distance euclidienne → élevée (à cause longueur)
- distance cosinus → proche (même orientation)

- **Similarité cosinus** : mesure d’orientation
 - compare seulement l’orientation (angle)
 - proche si les vecteurs pointent dans la même direction



souvent préférée
pour les
embeddings texte

Algorithme "Force Brute" (brute force k-NN)

1. Comparer le vecteur requête q avec *tous* les vecteurs du corpus

On calcule une similarité entre q et chaque x_i

$$s_i = \text{similarité}(q, x_i) \text{ pour } i=1..N$$

2. Trier tous les scores

On ordonne les vecteurs par similarité décroissante (ou distance croissante)

3. Prendre les k meilleurs

On garde les indices des k plus proches voisins

Complexité

- Nombre d'opérations : nombre de vecteurs N
- Temps $\approx O(N \cdot d)$, où d est la dimension des vecteurs

👉 **Très bien pour de petits corpus**
jusqu'à quelques milliers

👉 **Deviens ingérable** pour de grands corpus
millions de vecteurs → trop lent

Appliquer k-NN à des embeddings

Problématique : encoder le corpus avant

Word2Vec / FastText

- Les vecteurs des mots sont **déjà pré-calculés**
- On peut appeler directement **most_similar**("roi")

- Coût computationnel non négligeable (surtout pour des corpus grands)
- Stockage nécessaire : conserver les embeddings si l'on veut réutiliser le corpus sans tout ré-encoder

CamemBERT / SBERT

- Ce sont des **encodeurs** : ils transforment le texte en vecteurs mais **ne stockent pas une table "mot → vecteur"**
- Avant d'appliquer k-NN, il faut d'abord **encoder tout le corpus** (mots ou phrases) afin de construire l'espace vectoriel de référence
- **Différence :**
 - CamemBERT → nécessite un pooling pour fabriquer les embeddings
 - SBERT → produit directement des embeddings de phrases optimisés pour la similarité

Implémentation classique

(FastText pre-entraîné, Gensim KeyedVectors)

- **most_similar()** : recherche en force brute dans le vocabulaire
- Similarité Cosinus comme métrique par défaut

0.843	roi
0.760	monarque
0.724	souverain
0.704	royaume
0.704	prince

```
# Exemple avec un modèle FastText pré-entraîné
# Chargement du modèle réduit local
MODEL = KeyedVectors.load("cc fr 100k kv")

# Mots les plus proches de "roi"
for mot, score in MODEL.most_similar("roi", topn=5):
    print(f"{score: 3f} {mot} lower()")
```

Implémentation k-NN force brute avec CamenBERT (phrases)

```
from transformers import AutoTokenizer,
CamembertModel

import torch, torch.nn.functional as F

tok = AutoTokenizer.from_pretrained("camembert-
base")

cam = CamembertModel.from_pretrained("camembert-
base") eval()

corpus = ["le roi vit dans le château",
          "la reine habite le palais",
          "le président visite la capitale"]

with torch.no_grad():
    # 1) Encoder le corpus + mean pooling
    T = tok(corpus, padding=True,
return_tensors="pt")
    H = cam(**T) last_hidden_state
    M = T["attention_mask"] unsqueeze(-1)
    E = F.normalize((H*M) sum(1)/M
sum(1), p=2, dim=1) # embeddings
phrases
```

2) Encoder la requête

```
q = F.normalize(
    cam(**tok(["Le roi est important "],
return_tensors="pt"))
    last_hidden_state mean(1), p=2,
    dim=1)
```

3) Similarité cosinus force brute

```
scores = (q @ E.T) squeeze(0)
topk = torch.topk(scores, k=3)
for i, sc in zip(topk.indices, topk
values):
    print(f"{sc: 3f} {corpus[i]}")
```



0.905	le roi vit dans le château
0.882	la reine habite le palais
0.875	le président visite la capitale

Implémentation k-NN force brute avec Sentence-BERT (phrases)

```
from sentence_transformers import
SentenceTransformer, util
import torch

# 1) Charger le modèle (embeddings de
phrases optimisés pour la similarité)

sbert = SentenceTransformer("sentence-
transformers/paraphrase-multilingual-MiniLM-L12-v2")
```

2) Encoder le corpus de phrases (normalisé L2)

```
corpus = ["le roi vit dans le château",
          "la reine habite le palais",
          "le président visite la capitale"]

E = sbert.encode(corpus, convert_to_tensor=True,
normalize_embeddings=True)
```



0.773	le roi vit dans le château
0.664	la reine habite le palais
0.424	le président visite la capitale

3) Encoder la requête

```
q = sbert.encode(["Le roi est
important "], convert_to_tensor=True,
normalize_embeddings=True)
```

4) calculer les similarités (force brute)

```
scores = util.cos_sim(q, E)[0]
# cosinus = produit scalaire car
normalisé
```

```
topk = torch.topk(scores, k=3)
```

Affichage formaté (score à 3 décimales)

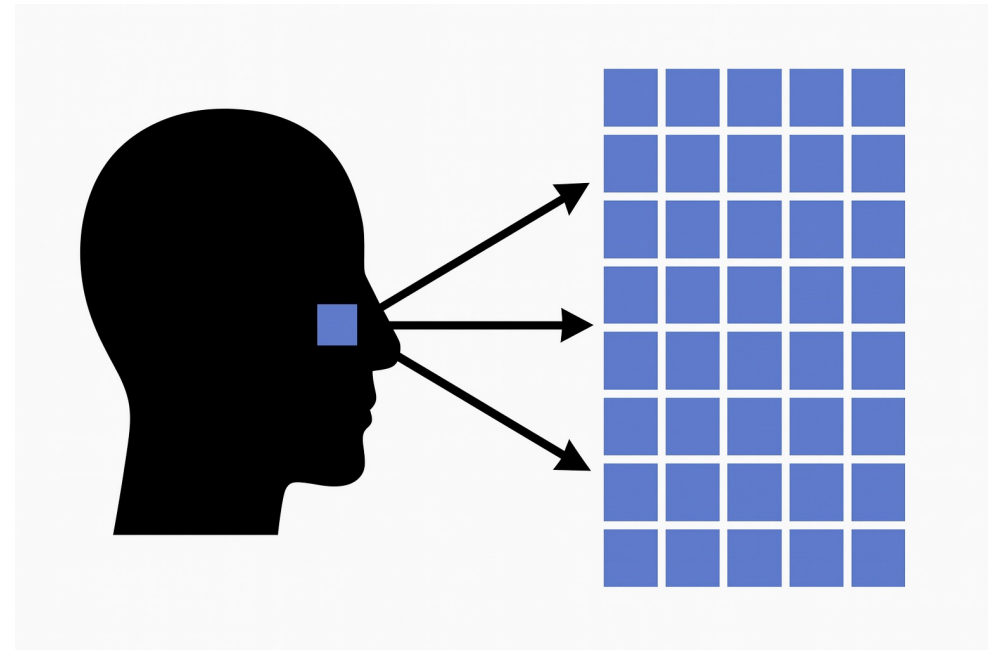
```
for i, sc in zip(topk.indices
tolist(), topk.values.tolist()):
    print(f"{sc: 3f} {corpus[i]}")
```

Limites du k-NN en force brute

■ Avantage : simple et intuitif

✗ Limites :

- Calcul coûteux pour de grands corpus (comparaison avec *tous* les vecteurs)
- Explosion en dimension (“malédiction de la dimensionnalité”)
- Pas adapté à la mise à jour dynamique de grands volumes



2 – Indexation vettoriale



Indexation vectorielle

Principes d'indexation vectorielle

- Objectif
- Bibliothèques d'indexation Vectorielles
- Tableau comparatif
- La bibliothèque FAISS

Index de recherche exacte

- Notion d'Index Flat
- Création dans FAISS
- Insertion de données
- Recherche dans un index

Index vectoriels ANN

- Principes des ANN
- IVF Inverted File Index : manipulation dans FAISS
- PQ (Product Quantization)
- HNSW (Hierarchical Navigable Small World Graph)

FAISS
Scalable Search With Facebook AI



Vers l'indexation vectorielle

Idée clé

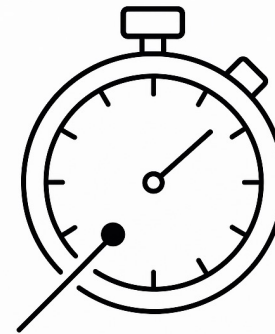


Plutôt que comparer la requête à tous les vecteurs →

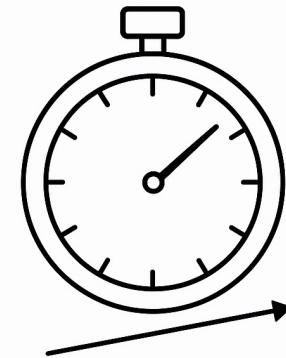
on organise l'espace pour accélérer la recherche

Principe

- Créer une structure d'index adaptée aux vecteurs
- Restreindre la recherche à une zone pertinente (clusters, arbres, graphes)
- Accepter une approximation → **gros gain de temps**



k-NN Force brute



ANN
(index structuré)

**Approximate Nearest
Neighbors (ANN)**



Les bibliothèques d'indexation vectorielle

Implémentent concrètement les algorithmes ANN (IVF, PQ, HNSW...)

🎯 Objectif : **accélérer la recherche de similarité** dans les espaces vectoriels

❌ Pas de gestion complète de BD → juste un **moteur d'index**

🌍 La plus utilisée : FAISS (Meta AI)



Tableau comparatif des librairies d'indexation

Librairie	Langage / Implémentation	Forces	Limites
FAISS (Meta)	C++/CUDA + Python	Très rapide, GPU, nombreux index (IVF, PQ, HNSW)	Installation parfois lourde, pas de persistance
Annoy (Spotify)	C++ + Python	Simple, léger, faible mémoire	Index statique (pas d'update), précision moindre
HNSWlib	C++ + Python	Implémentation HNSW optimisée, excellent compromis vitesse/précision	Moins de variété d'index que FAISS
ScaNN (Google)	C++ + Python	Optimisé Google-scale, TensorFlow-friendly	Moins documenté, plus complexe à installer

FAISS : Facebook AI Similarity Search



Librairie open-source (Meta AI) depuis 2017

Doc officielle : <https://faiss.ai/index.html>



Optimisée C++/CUDA, interface Python



Objectif : recherche de vecteurs à grande échelle



Une boîte à outil d'index

Recherche exacte (*IndexFlat*)

Recherche approximative (*ANN*)

VF, PQ, HNSW, etc



Cas d'usage : NLP (embeddings de phrases), Vision (features d'images), Reco systèmes, RAG



*Pour passer de la
recherche brute force à
l'ANN scalable*

Index de recherche exacte : FLAT

(🔨 Force brute optimisée)

▢ Principe : stocke tous les vecteurs, recherche par produit scalaire / distance euclidienne

✅ Avantages :

Même si l'algorithme est "force brute"

- Résultats exactement identiques à la recherche naïve
- Beaucoup plus rapide qu'une implémentation naïve Python (optimisations C++/SIMD)

❌ Limite : temps linéaire $O(N)$, pas scalable à très grande échelle
Exemple d'usage : petit corpus (< 100k vecteurs)

👉 **Point de référence** pour évaluer les autres index ANN (IVF, PQ, HNSW).

Les index exacts Flat dans FAISS

■ IndexFlatL2(**d**)

d = taille des vecteurs

→ calcule les distances euclidiennes (L2)

→ utile quand la taille des vecteurs est importante : ex données géographiques

■ IndexFlatIP(**d**)

d = taille des vecteurs

→ calcule le produit scalaire (*inner product*)

→ après normalisation L2, équivaut à la similarité cosinus

👍 Le choix naturel pour des comparaison sémantiques (embeddings) mais il faut normaliser les vecteurs avant l'indexation

Etapes de création et manipulation d'un index Flat

Créer l'index



`index = faiss.IndexFlatIP(d)`



Ajouter des
vecteurs



`index.add(E)`



Rechercher



`D, I = index.search(QX, k)`

➡ `d` = dimension des
vecteurs (nb de
colonnes)

➡ `E` = matrice
`numpy(N,d)`
`N` = nb de vecteurs
`d` = dimension des
vecteurs

➡ `QX` = matrice `numpy(Q,d)`
`Q` = nb de requêtes
`d` = dimension des vecteurs
➡ `k` = nb de voisins cherchés
➡ `D` et `I` matrices(`Q,k`)
`D` = scores de similarites
`I` = indices des vecteurs `k`-voisins

Création d'un index Flat avec FAISS

```
import faiss
#creation de l'index
index = faiss.IndexFlatIP(364)
# ou index = faiss.IndexFlatL2(d)
#ajout de vecteurs
index.add(E)      # E : matrice numpy (N, 364), float32
# affichage nombre de vecteurs stockés
print(index.ntotal)
```

chaque ligne de E est un vecteur (embedding)

- ⚠ E Nécessite des float32
- Possibilité de normaliser avant (cosinus $\leftrightarrow$ produit scalaire)

Requêtes de Recherche

`D, I = index.search(q, k)`

- `q` : matrice des requêtes de taille (nq, d)
 - nq = nombre de requêtes (souvent 1 pour un test simple)
 - d = dimension des vecteurs
- k : nombre de voisins que l'on veut récupérer

matrice requete

q

0	0.8	0.2	0.0
1	0.5	0.1	0.4



Résultats renvoyés

nombre de lignes =
nombres de requetes

- **I (indices)**
 - matrice (nq, k)
 - $I[i, j]$ = indice (dans l'index) du j -ème voisin le plus proche de la requête $q[i]$
 - permet de savoir *quel* vecteur a été retrouvé
- **D (distances / similarités)**
 - matrice (nq, k)
 - $D[i, j]$ = score associé au voisin $I[i, j]$

Exemple de Requête de Recherche de similarité

```
# --- Corpus de 3 "couleurs" en dimension 3 (R, G, B) ---
E = np.array([
    [1.0, 0.0, 0.0],    # Rouge pur
    [0.0, 1.0, 0.0],    # Vert pur
    [0.0, 0.0, 1.0],    # Bleu pur
], dtype='float32')
# --- Création d'un index Flat IP (produit scalaire) dim=3---
index = faiss.IndexFlatIP(3)
index.add(E)
# --- Requête : une couleur orangée = mélange de rouge et un peu de vert
q = np.array([[0.8, 0.2, 0.0]], dtype='float32')
# --- Recherche top-2 voisins ---
D, I = index.search(q, k=2)
print("Indices des voisins:", I)
print("Scores (produit scalaire):", D)
```

attention: search attend une matrice 2D: ajout [...]

Requêtes de Recherche de similarité - exemple

E

rouge	0	1.0	0.0	0.0
vert	1	0.0	1.0	0.0
bleu	2	0.0	0.0	1.0

Requête $q = (0.8, 0.2, 0.0)$

- Produit scalaire avec **Rouge** (1,0,0) = 0.8

- Produit scalaire avec **Vert** (0,1,0) = 0.2

- Produit scalaire avec **Bleu** (0,0,1) = 0.0

👉 Les scores sont clairs : Rouge (0.8) > Vert (0.2) > Bleu (0.0).

D

0	0.8	0.2
---	-----	-----

score similarité avec le vecteur 0 rouge
(q,X[0])

q matrice requete

0	0.8	0.2	0.0
---	-----	-----	-----

#creation index

index = faiss.IndexFlatIP(3)

index.add(E)

#requete

D, I = index.search(q, k=2)

Plus les scores sont élevés = plus les vecteurs sont proches

I

0	0	1
---	---	---

indice du voisin N°0

D, I =

- 1 seule ligne car q a une ligne
- 2 colonnes car k=2

indice du voisin N°1

Exemple de requete de proximité géographique

```
# Positions 3D de capteurs (x, y, z)
```

```
positions = np.array([
    [0.0, 0.0, 0.0], # capteur A
    [1.0, 0.0, 0.0], # capteur B
    [0.0, 1.0, 0.0], # capteur C
    [2.0, 1.0, 0.0], # capteur D
], dtype='float32')
```

```
# --- Création d'un index Flat L2 (distance euclidienne) dim=3---
```

```
index = faiss.IndexFlatL2(3)
index.add(E)
```

```
# --- Requête : un point d'interet à localiser
```

```
q = np.array([[1.2, 0.7, 0.0]], dtype='float32')
```

```
# --- Recherche top-2 voisins ---
```

```
D, I = index.search(q, k=2)
```

```
print("Indices des capteurs les plus proches:", I)
```

```
print("Distances par rapport à ces capteurs :", D)
```

Indices des capteurs les plus proches: `[[1 3]]`

Distances par rapport à ces capteurs : `[[0.53            0.72999996]]`

Plus les distances
sont faibles = plus les
vecteurs sont
proches

Normalisation et métriques

```
faiss.normalize_L2(X)  
faiss.normalize_L2(q)
```

- X : matrice numPy
- Forme (n, d)
 - n = nombre de vecteurs
 - d = dimension des vecteurs
- Normalise chaque vecteur à norme L2 = 1
 - Après normalisation :
 - Produit scalaire = cosinus
 - Plus simple pour comparer avec brute force

Obligatoire avant Add si on utilise IndexFlatIP
→ le produit scalaire devient exactement la similarité cosinus

Exemple avec et sans normalisation

```
# SANS normalisation (IP ≠ cosinus)
```

```
index = faiss.IndexFlatIP(3)
index.add(E)
D, I = index.search(q, k=3)

print("Voisins (sans normalisation)")
print("Indices: ", I, "Scores (produit scalaire):", D)
```

```
Voisins (sans normalisation)
Indices:  [[0 1]] Scores (produit scalaire): [[0.8 0.2]]
```

les similarités ne correspondent pas aux similarités cosinus qque si les vecteurs sont normalisés

```
# AVEC normalisation (IP == cosinus)
```

```
# normalise chaque ligne à ||x||=1
```

```
E_norm = E.copy()
q_norm = q.copy()
faiss.normalize_L2(E_norm)
faiss.normalize_L2(q_norm)
index_norm = faiss.IndexFlatIP(3)
index_norm.add(E_norm)
D_norm, I_norm = index_norm.search(q_norm, k=2)
print("Voisins(avec normalisation)")
print("Indices: ", I_norm, "Scores (similarité cosinus):", D_norm)
```

```
Voisins(avec normalisation)
Indices:  [[0 1]] Scores (similarité cosinus): [[0.9701425 0.24253562]]
```

Idée des recherches approximatives (ANN : Approximate Nearest Neighbors)

Pourquoi passer à l'approximation ?

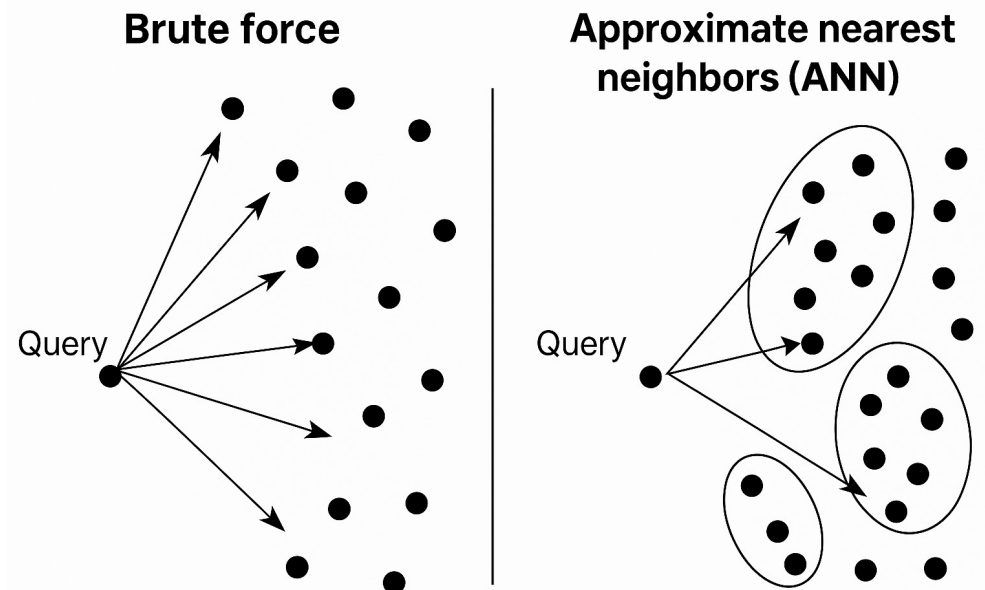
✗ Recherche exacte Index FLAT
(Force brute)

Trop lente quand N devient grand
(millions de vecteurs)

■ Idée clé de l'ANN



- Ne pas parcourir tout le corpus → explorer seulement une partie prometteuse



Principe des ANNs

Construire un index structuré (ex clusters, graphe, quantification)

La requête interroge seulement une fraction de l'index

Résultat

- ⚡ Recherche beaucoup plus rapide
- ✅ Qualité souvent très proche de la recherche exacte
- ❌ Possibilité de rater certains voisins $\rightarrow$ rappel < 1

Message clé

👉 L'ANN repose sur un **compromis vitesse $\leftrightarrow$ qualité** :

Plus on réduit l'exploration $\rightarrow$ plus rapide, mais moins précis

Plus on augmente l'exploration $\rightarrow$ plus de précision, mais temps de calcul $\uparrow$

Comment évaluer les résultats d'un ANN ?

Recall@k

1 Rappel (Recall@k)

Proportions des vrais voisins (oracle exact) parmi les k voisins renvoyés par l'index ANN

Formule :

nombre de bons voisins trouvés



$$Recall@k = \frac{|Top_k^{exact} \cap Top_k^{ANN}|}{k}$$

Valeur entre **0** et **1** :

1 = parfait

0 = aucun bon voisin retrouvé

Recall = capacité à retrouver tous les bons voisins par rapport à la recherche exacte



Exemple simple

Corpus = 1000 vecteurs

Requête q

$Top_3^{exact} = \{A, B, C\}$ **#oracle exact**

$Top_3^{ANN} = \{A, D, E\}$ **#résultat ANN**



Intersection = {A} → taille = 1

$$Recall@3 = \frac{1}{3} = 0,33$$

1 seul vrai voisin exact a été trouvé

recall@k : exemple d'implémentation

→ $Top_3^{exact} = \text{true_idx}[:5] = [10, 42, 7, 88, 35]$

indices des k vrais voisins (trouvés par IndexFlatIP par ex)

→ $Top_3^{ANN} = \text{approx_idx}[:5] = [10, 99, 42, 35, 77]$

indices des k voisins renvoyés par l'index ANN (ex: IVF)

Intersection = {10, 42, 35} (3 éléments)

■ $Recall@5 = \frac{3}{5} = 0.6$

3 vrais voisins exacts ont été trouvés

```
def recall_at_k(true_idx, approx_idx, k=5):  
    return len(set(true_idx[:k]).intersection(  
        set(approx_idx[:k]))) / k
```

set transforme les listes en ensemble (élimine les doublons)

Comment évaluer les résultats d'un ANN ?

Autres mesures

2 Temps de réponse d'une requête : latence

Mesurer la latence moyenne d'une requête



Objectif :

garder des temps < 100 ms en production

```
#mesurer la latence d'une requête  
t0 = time.perf_counter()  
D, I = index.search(q, k)  
latence = time.perf_counter() - t0
```

3 Mémoire et taille de l'index

Vérifier le coût mémoire selon l'index choisi

Vers l'amélioration

Ajuster les paramètres (nprobe, efSearch, etc) pour trouver le meilleur équilibre selon l'application

Les principaux types d'index ANN



IVF

Partitionne l'espace
en clusters
(quantification)



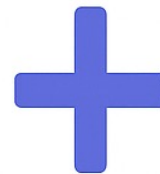
HNSW

Graphe de proximité
navigable



PQ

Compression des vecteurs
par sous-espaces



Combinaisons

Équilibre entre
vitesse, mémoire et
précision

Chaque index a ses **avantages / limites** selon :

- la taille du corpus - la précision souhaitée - les ressources disponibles

Index IVF (Inverted File Index)

Principe

1- Construction des clusters

quantizer + train

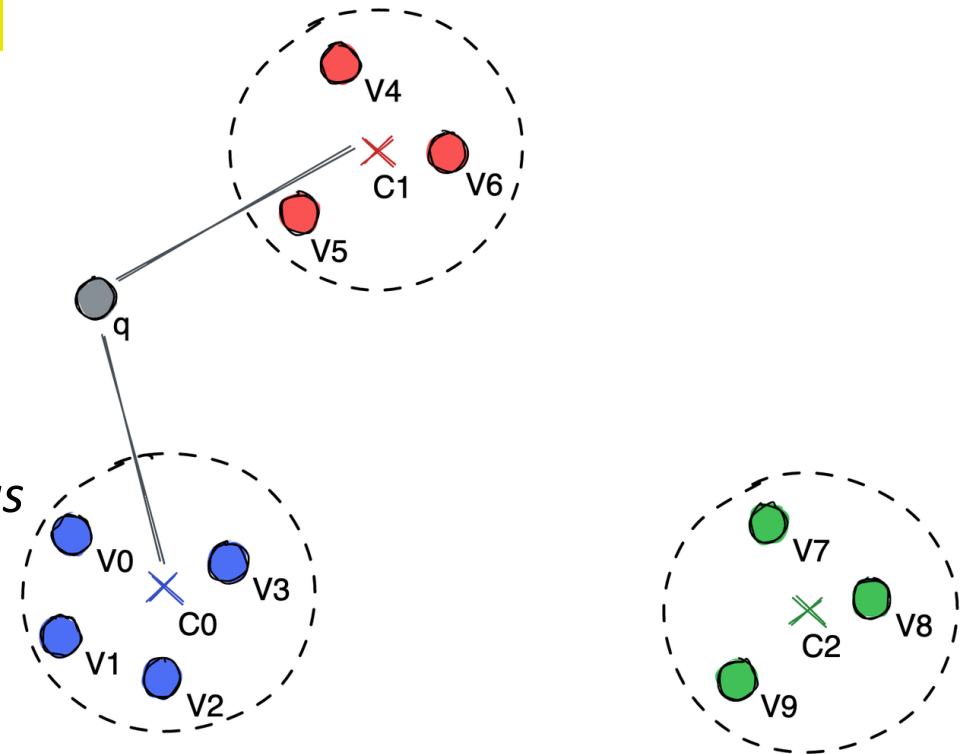
- Diviser les vecteurs en *clusters* (k-means)
- Chaque cluster est représenté par un **centroïde** (moyenne des points)

2- Recherche

- Comparer la requête q aux centroïdes
- Explorer uniquement les ***nprobe*** clusters les plus proches

✓ **Avantage** : vitesse (scan partiel du corpus)

✗ **Limite** : approximation (rappel < 1 si *nprobe* petit)



Des points divisés en groupes représentés par un centroïde



Paramètres clés de l'index IVF

nlist = nombre de clusters

Plus il est grand → index plus fin, meilleure précision

Mais temps de construction et mémoire ↑

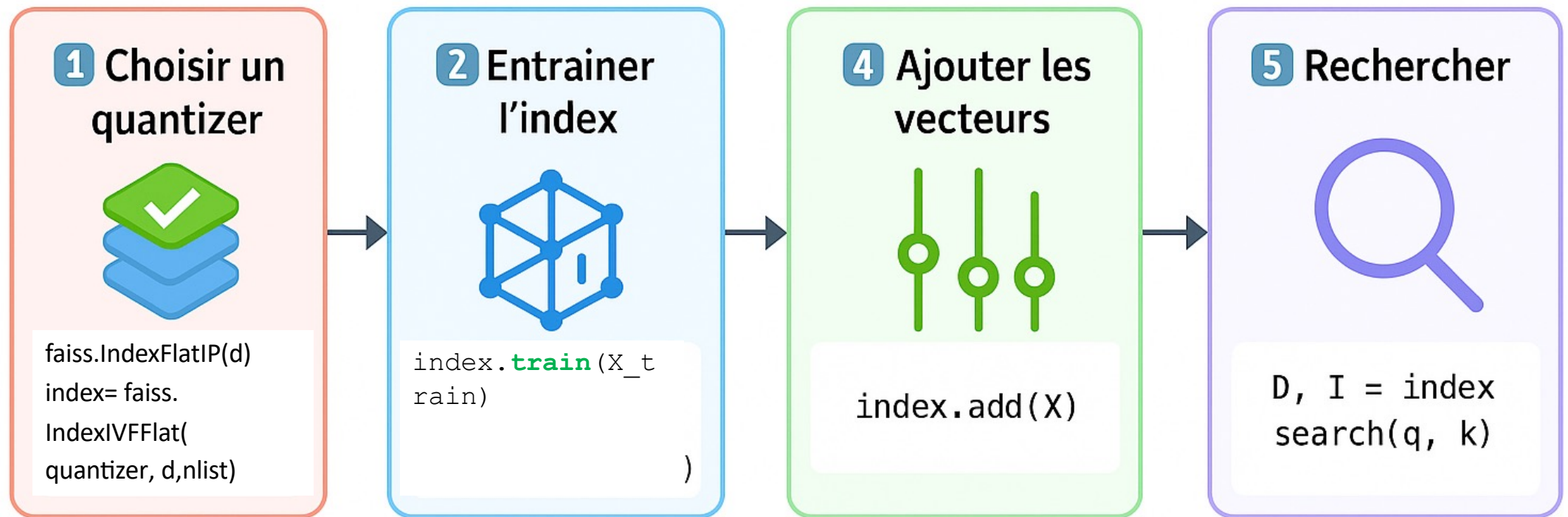
nprobe = nombre de clusters explorés lors d'une requête

Petit → recherche rapide mais risque de manquer des voisins (rappel ↓)

Grand → meilleure qualité mais latence ↑

👉 Le choix de *nlist* et *nprobe* permet d'ajuster le **compromis vitesse ↔ qualité**

Étapes de construction d'un Index IVF (FAISS)



- d: dimension des vecteurs
- nlist : nombre de clusters
- X-train= matrice des vecteurs d'entrainement (peut être X si X petit)

Index IVF dans FAISS

IndexIVFFlat

1 - Choisir un quantizer : un IndexFlat sert de base au clustering

```
quantizer = faiss.IndexFlatIP(d) # ou L2
```

- d: dimension des vecteurs
- nlist : nombre de clusters

2- créer l'index IDF

```
index = faiss.IndexIVFFlat(quantizer, d, nlist,  
faiss.METRIC_INNER_PRODUCT)
```

mesure par produit scalaire , faiss. METRIC_L2
par défaut (distance euclidienne) sinon

```
index.train(X_train) # 3- lance le clustering k-means
```

```
index.add(X) # 4- assigne chaque vecteur à un centroïde
```

```
index.nprobe = 8 # 5 - choix nb de listes explorées
```

```
D, I = index.search(q, k)
```

la matrice d'entrainement X-train

- Forme : (N_train, d) où :
 - N_train = nombre de vecteurs utilisés pour l'entraînement,
 - d = dimension des vecteurs (même que celle de l'index).-
- Le rôle de X_train est uniquement de servir à la phase **d'apprentissage des nlist centroïdes**
- On peut utiliser un **sous-échantillon représentatif** (par exemple 10 000 vecteurs choisis aléatoirement) pour entraîner l'index

On peut aussi utiliser le corpus entier (cf. TP3)

PQ (Product Quantization)

▢ Principe : compresse chaque vecteur en sous-codes via quantification

✅ Avantage : réduit la mémoire (stockage compact), accélère la recherche

❌ Limite : approximation plus forte (perte de précision)

Usage : très grands corpus où la mémoire est critique

👉 *Quand le volume explose, PQ permet de stocker des milliards de vecteurs*

PQ (Product Quantization): Exemple



Découpage

Un vecteur de grande dimension (ex 128D) est séparé en sous-vecteurs plus petits (ex 8 blocs de 16D)



Quantification

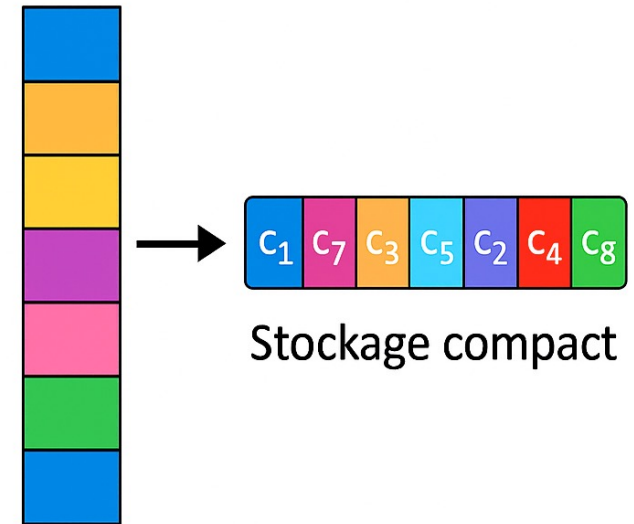
Dans chaque sous-espace, on applique un *k-means* → chaque sous-vecteur est remplacé par l'indice du centroïde le plus proche dans la table de quantification



Exemple

Vecteur initial : [0 12, -0 87, 0 33, ...] (128 flottants)

Après PQ : [c1, c7, c3, c5, c2, c9, c4, c8] (8 indices seulement)



👉 Résultat 💾

Stockage réduit d'un facteur 10 à 20, avec une perte de précision contrôlée

HNSW

(Hierarchical Navigable Small World Graphs)

▢ Principe

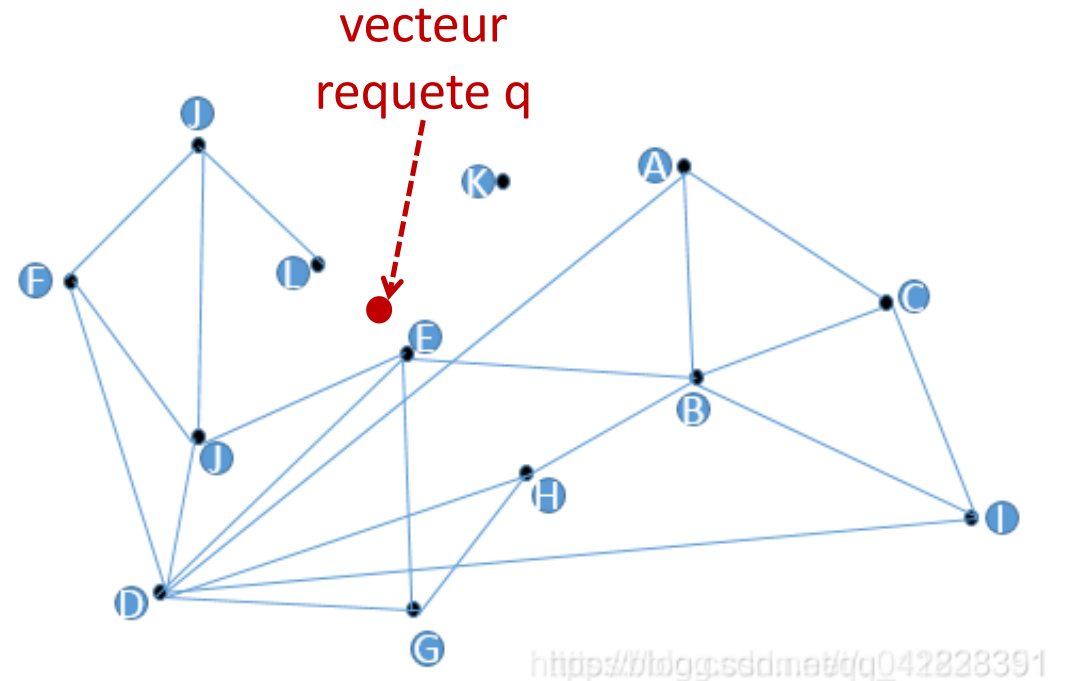
- Construit un graphe de proximité entre vecteurs
- La recherche suit des arêtes dans le graphe (parcours hiérarchique)

✅ Avantage : très rapide, excellent rappel

❌ Limite : index plus lourd, mémoire ↑

👉 Usage

- Souvent choisi par défaut pour des ANN haute qualité
- ⚖️ HNSW = équilibre **vitesse + précision**, très utilisé en production

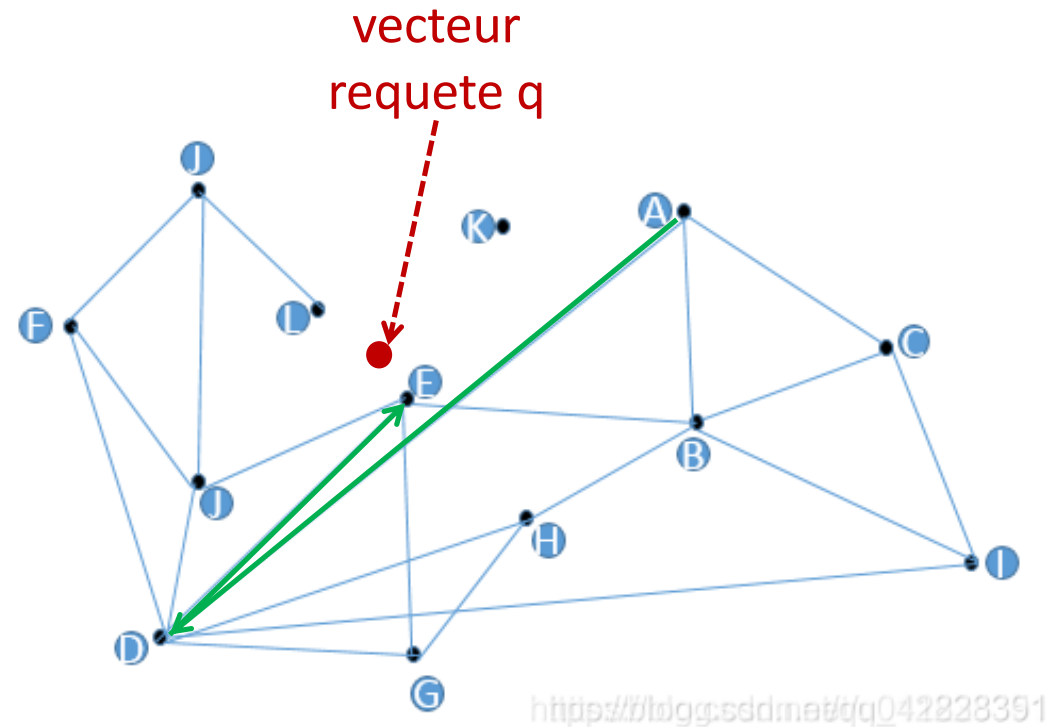


HNSW : Exemple de parcours

1. **Départ** : choisir au hasard un sommet d'entrée (ex A)
2. **Parcours** : avancer vers le voisin le plus proche de $q \rightarrow A \rightarrow D \rightarrow E$
3. **Arrêt** : tous les voisins de E sont plus éloignés $\rightarrow$ E est le plus proche voisin

⚠ Limites

- Certains sommets peuvent être inaccessibles (ex: K)
- Recherche locale parfois coûteuse



INDEX Vectoriels : Résumé comparatif

Index	Exact / Approx	Avantage clé	Limite	Cas d'usage typique
Flat	Exact	Simplicité, exactitude	Linéaire, pas scalable	<100k vecteurs
IVF	Approx	Vitesse, paramétrable	Rappel < 1	Millions de vecteurs
PQ	Approx	Compression mémoire	Perte de précision	Milliards de vecteurs
HNSW	Approx	Vitesse + haute qualité	Index lourd	ANN production

A vous de jouer : TP3_IndexationFAISS.pynb

- Récupérez le notebook TP3_IndexationFAISS.pynb

Partie A — Comparaison Force brute et FAISS Index Flat

Partie B — Indexation avec FAISS-IVF (Inverted File Index)



3 – SGBD vectoriels



BD Vectorielles

Principes des SGBD vectoriels

- Librairie d'indexation VS SGBD vectoriel
- Métadonnées dans les BD vectorielles
- Etapes d'utilisation d'une BD vectorielle
- Evaluation de la pertinence d'une recherche

SGBD vectoriels

- Principaux SGBD Vectoriels
- Comparatif

ChromaDB

- Fonctionnalités principales
- Workflow
- Principales manipulations
- "Sous le capot" : principes
- Exemple guidé

Des librairies d'indexation aux SGBD vectoriels

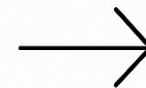
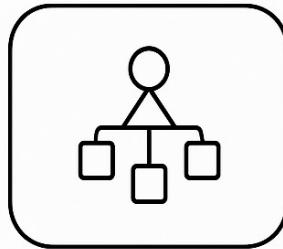
Librairies d'indexation

👉 moteur d'index local

Algorithmes performants pour la **recherche de voisins proches (ANN)**

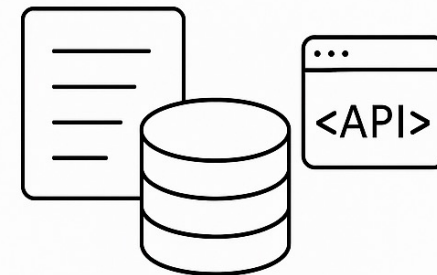
- ❌ Pas de stockage persistant
- ❌ Pas de gestion de métadonnées
- ❌ Pas d'API de requêtes ni de distribution

FAISS, Annoy, HNSWlib



SGBD vectoriels

👉 une infrastructure de données prête pour la production

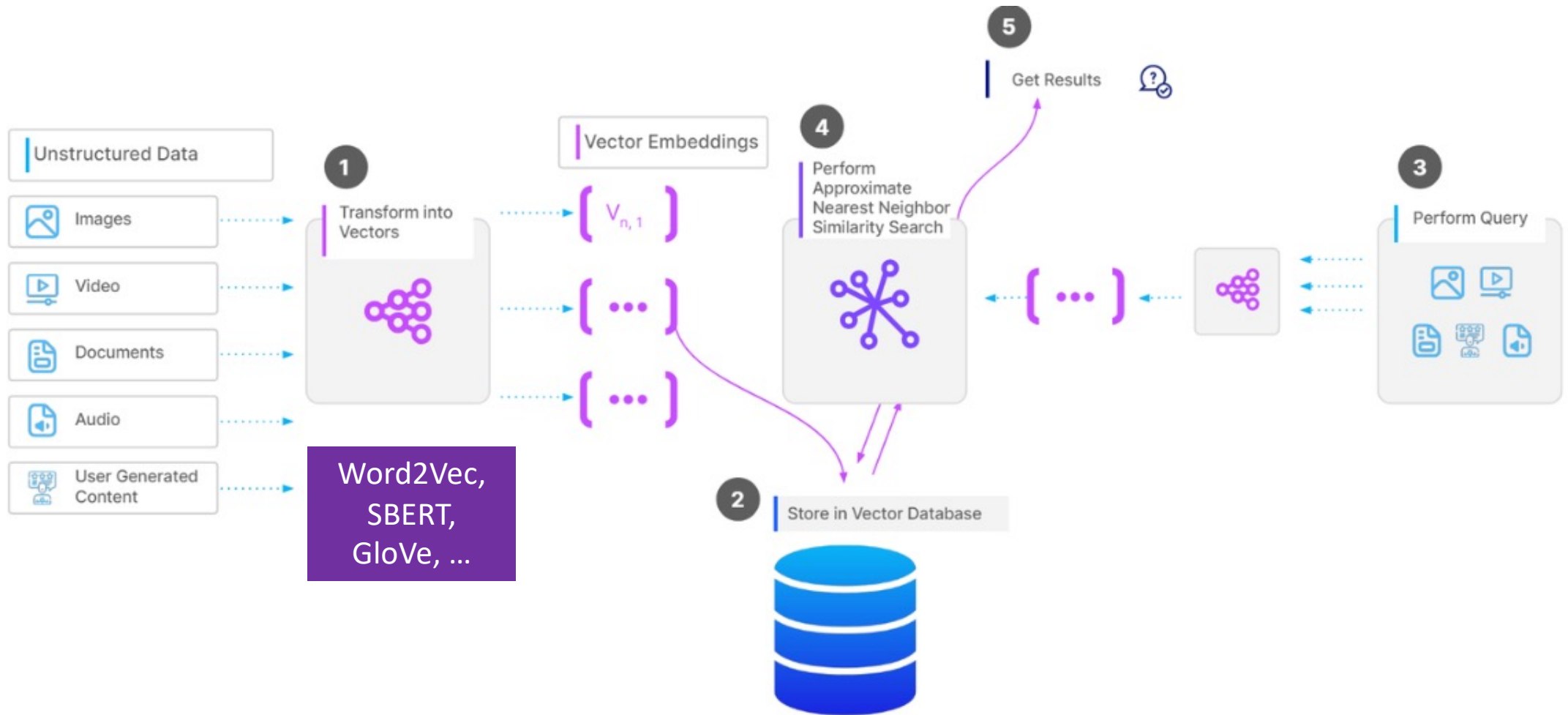


Qu'est-ce qu'un SGBD vectoriel ?

- SGBD spécialisé pour les embeddings
- Fonctionnalités
 - 💾 Stockage persistant des vecteurs
 - ⚡ Index ANN intégré
 - 📝 Métadonnées hybrides
(texte + vecteur)
 - 🔍 Requêtes unifiées
(SQL + recherche vectorielle)



Workflow d'une BD vectorielle : de l'encodage à la recherche



Notion de Métadonnées dans les BD vectorielles

- Informations associées aux embeddings stockés
- Ne participent pas directement à la similarité vectorielle
- Utiles pour filtrer et organiser
 -  **Filtrage** : chercher uniquement dans une source donnée
 -  **Combinaison** : recherche vectorielle + conditions sur métadonnées
 -  **Organisation** : regrouper les résultats par catégorie, auteur, période

Exemples



Source : nom du document ou URL



Date : de création ou d'indexation



Auteur : personne, organisation



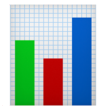
Catégorie / tag : thème, domaine, type de contenu



Contexte : chapitre, paragraphe, langue

Totalement personnalisables
et flexibles dans Chroma

Les métadonnées enrichissent les embeddings en ajoutant un contexte structuré indispensable pour un RAG



Évaluer la pertinence d'une recherche (côté utilisateur)

- **Problématique**

Une base vectorielle retrouve des voisins proches, mais sont-ils pertinents pour l'utilisateur ?

- **Qui juge la pertinence ?**

L'utilisateur (ou un annotateur) indique quels documents sont réellement pertinents



- **Comment procéder concrètement ?**



calcul de **métriques** qui comparent les résultats renvoyés par la base $\leftrightarrow$ les documents étiquetés pertinents

Métriques classiques IR (Information Retrieval)

Précision@k

$$= \frac{\#docs\ pertinents\ retrouvés\ dans\ le\ top-k}{k}$$

👉 *Qualité des résultats proposés*

Rappel@k (Recall@k)

$$= \frac{\#docs\ pertinents\ retrouvés\ dans\ le\ top-k}{\#docs\ pertinents\ existants}$$

👉 *Couverture des résultats pertinents*



Exemple simple (catégorie = "animaux")

- **Resultat obtenu (requete Top-3** = {2 docs "animaux", 1 doc "économie"}
- Pertinents existants : 5 docs "animaux"
- **Précision@3** = $2/3 \approx 0.67$
- **Rappel@3** = $2/5 = 0.40$

- ✓ Précision → **justesse** des résultats
- ✓ Rappel → **complétude** des résultats pertinents

Recall@k

$$= \frac{\text{docs pertinents retrouvés}}{\text{docs pertinents existants}}$$

Recherche Sémantique

- Embeddings + FAISS
- pertinence de contenu



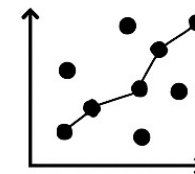
de documents pertinents
retrouvés dans le top-k

Evaluer la pertinence de la
recherche



couverture des
résultats pertinents

Index ANN



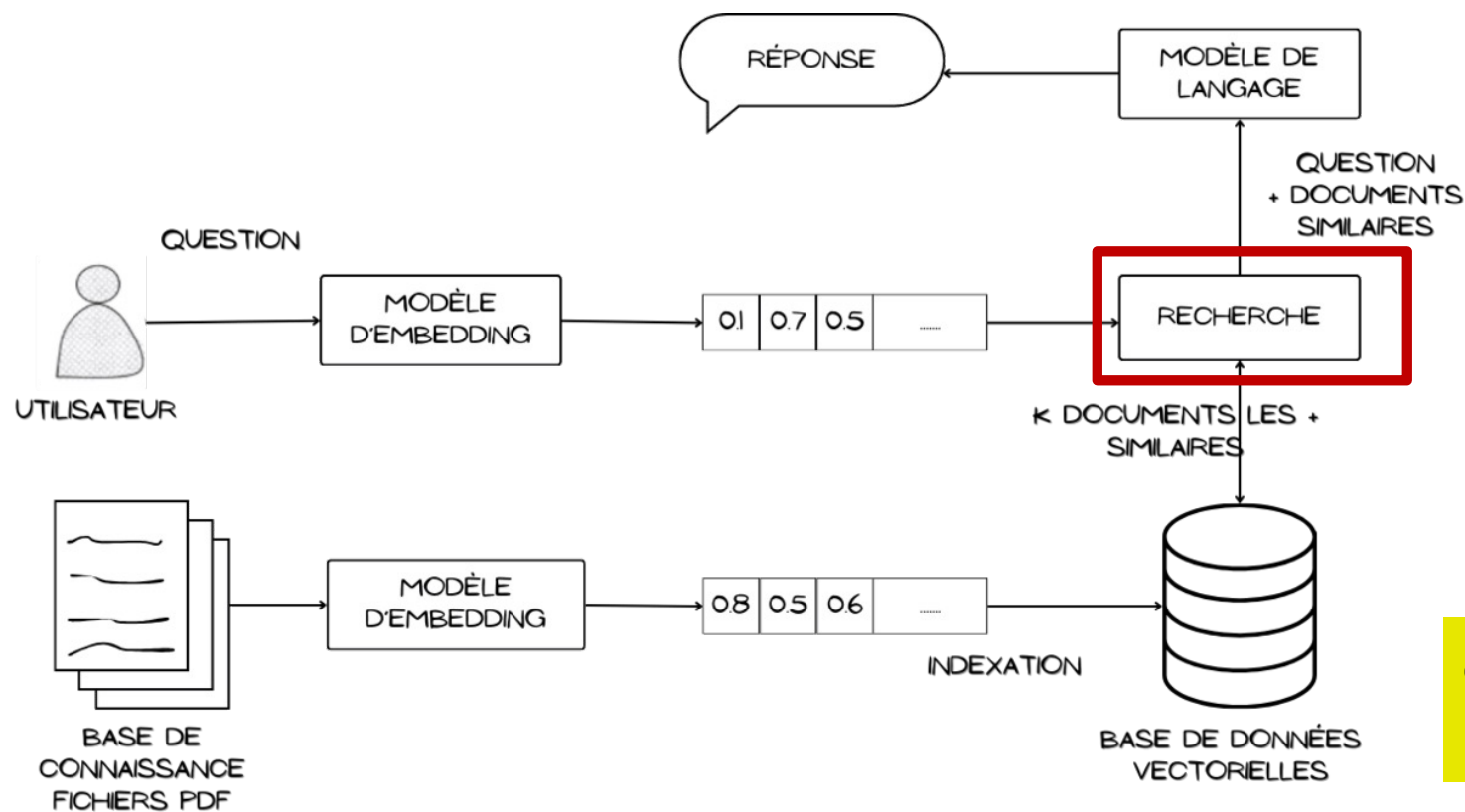
similarité géométrique
dans l'espace vectoriel

Evaluer la fiabilité de l'index

$$\frac{\text{nombre de bon voisins trouvés}}{\text{nombre de voisins cherchés } k}$$

Intégration dans l'écosystème IA

Embeddings et RAG



cf. partie II du cours

Principaux SGBD vectoriels

☐ include secondary database models 22 systems in ranking, August 2025

Rank			DBMS	Database Model	Score		
Aug 2025	Jul 2025	Aug 2024			Aug 2025	Jul 2025	Aug 2024
1.	1.	1.	Elasticsearch	Multi-model 	114.27	-4.56	-15.56
2.	2.	2.	OpenSearch	Multi-model 	18.48	+1.18	+2.01
3.	3.	3.	Couchbase	Multi-model 	12.67	-0.29	-3.53
4.	4.	4.	Kdb	Multi-model 	6.98	-0.12	-0.99
5.	5.	↑ 6.	Pinecone	Vector	4.95	+0.66	+1.81
6.	6.	↓ 5.	Milvus	Vector	3.83	+0.47	+0.45
7.	7.	↑ 10.	Qdrant	Vector	2.92	+0.39	+1.54
8.	8.	↑ 9.	OceanBase	Multi-model 	2.82	+0.31	+0.93
9.	9.	↑ 11.	Weaviate	Vector	2.28	+0.13	+0.99
10.	↑ 11.	↓ 8.	Chroma	Vector	2.22	+0.21	+0.33

Fonctionnalités vectorielles dans un outil généraliste

SGBD spécialisés

source : <https://db-engines.com/en/ranking/vector+dbms>



open source

<https://www.trychroma.com/>



Milvus

apparu en 2019



Weaviate



Pinecone



Qdrant

Comparatif des SGBD vectoriels

SGBD vectoriel	Type	Points forts	Cas d'usage privilégiés
Milvus	Open source, distribué	Scalabilité, moteur ANN intégré, support hybride (SQL + vecteurs), communauté active	Applications industrielles, cloud & on-premise
Weaviate	Open source, orienté graphe	API GraphQL, intégration native LLM/transformers, plug-ins	RAG, moteurs de recherche sémantique, knowledge graph
Chroma	Open source , léger, Python natif	Simple à intégrer, persistance locale, très utilisé avec LangChain	Prototypage RAG, projets académiques et légers
Qdrant	Open source (Rust), distribué	Haute performance, moteur HNSW natif, support filtering + payloads	Recherche sémantique temps réel, systèmes recommandation
Pinecone	SaaS (propriétaire)	Cloud natif, scalabilité automatique, faible maintenance	Déploiement industriel clé en main, production massive

Chroma DB : un SGBD vectoriel simple et léger

🟢 **Open-source**, Python natif



⚙️ Installation facile : `pip install chromadb`

📁 Création et gestion de **bases vectorielles (collections)**

💾 Stockage **persistant** (local ou en mémoire)

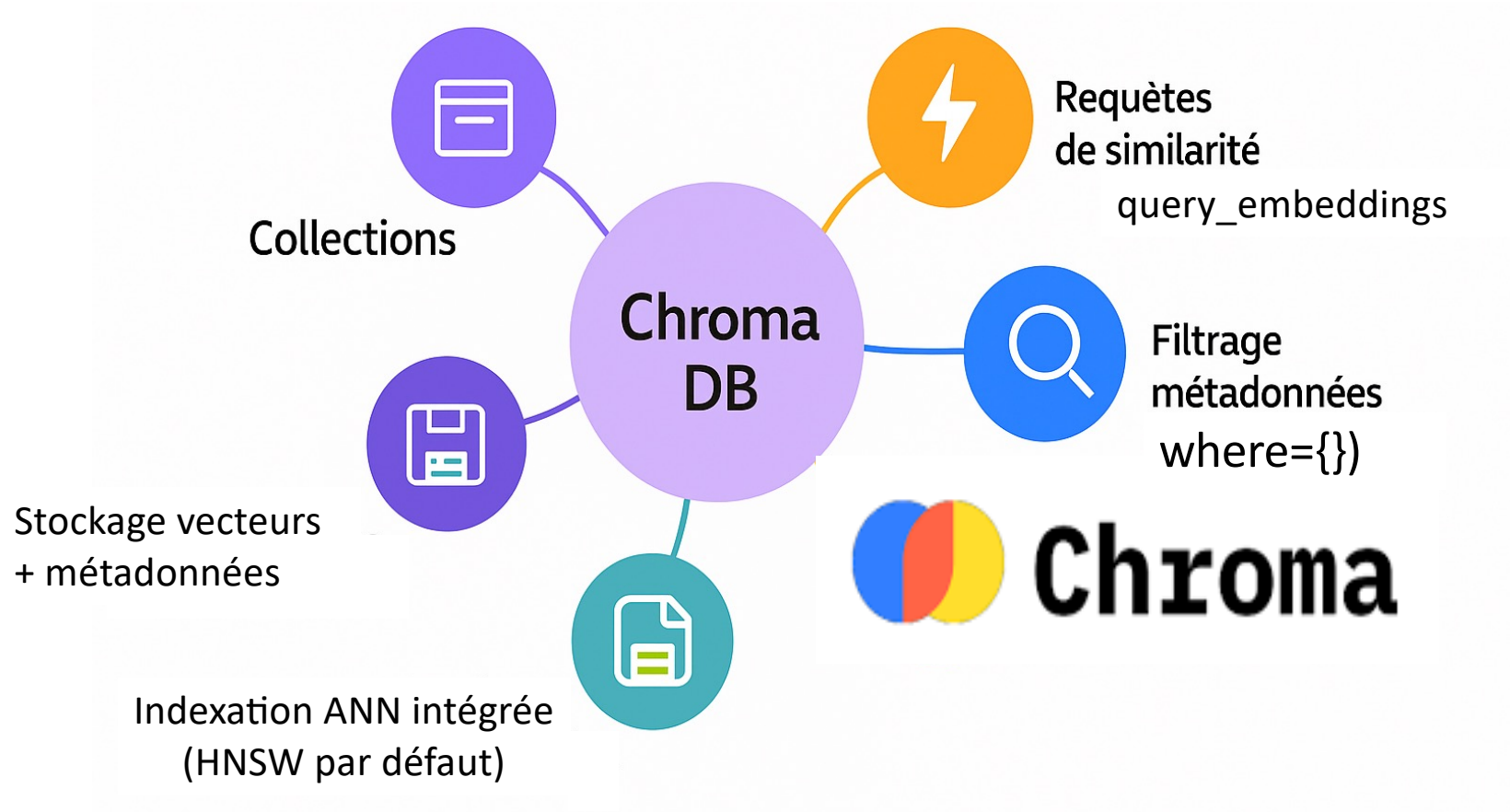
🔗 Intégration avec **LangChain** / **LlamaIndex** → très utilisé en RAG

🎓 ➡️ Idéal pour l'enseignement, les TP et le prototypage rapide

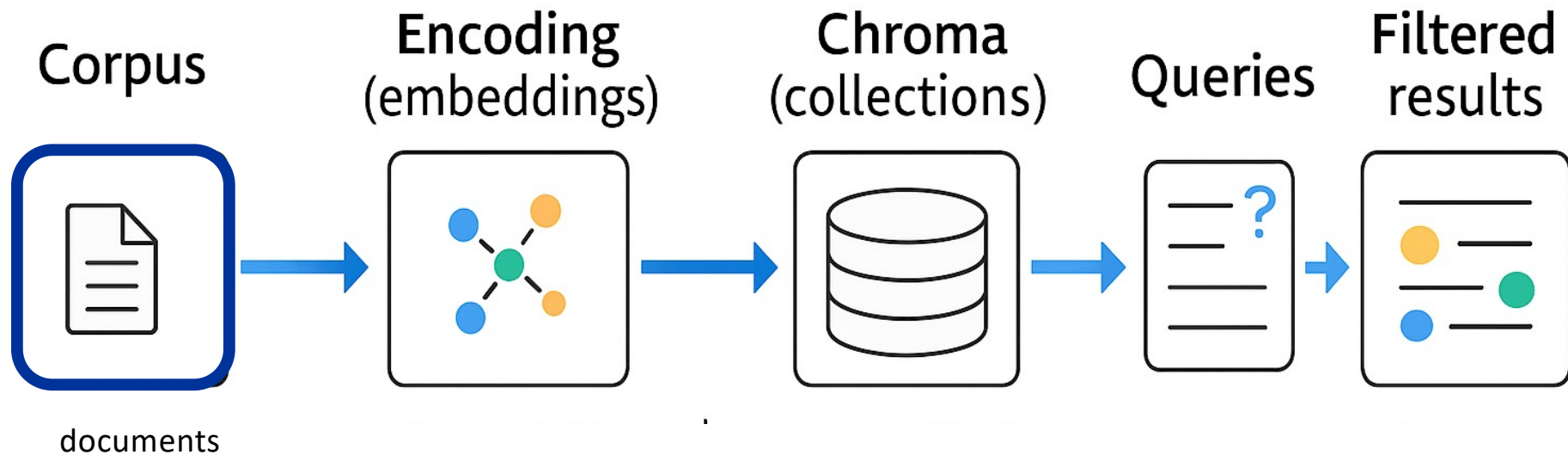
🔄 Passer à Milvus, Weaviate, Qdrant, Pinecone

Pour des projets plus lourds, distribués, scalables et orientés production

Chroma: fonctionnalités principales



Workflow Chroma



Exemples du cours à tester dans le notebook du cours
TPCours-BDVectorielleChroma.pynb

Initialisation du SGBD

Création du client Chroma persistant

```
import chromadb
from chromadb.config import Settings

# Client SGBD (persistance locale)
client = chromadb.Client(Settings(
    is_persistent=True,
    persist_directory="chroma_data"
))
```



Dossier de sauvegarde
locale de la BD

Structure d'une collection Chroma



Une collection Chroma = 4 composants principaux

Élément	Rôle	Exemple
ids	identifiants uniques des vecteurs	"doc_0", "doc_1"
documents	textes d'origine	"Le chat dort sur le canapé"
embeddings	vecteurs denses (représentation sémantique)	[0.12, -0.47, ...]
metadatas	infos associées (catégorie, source, auteur...)	{"cat":"animaux","source":"blog"}

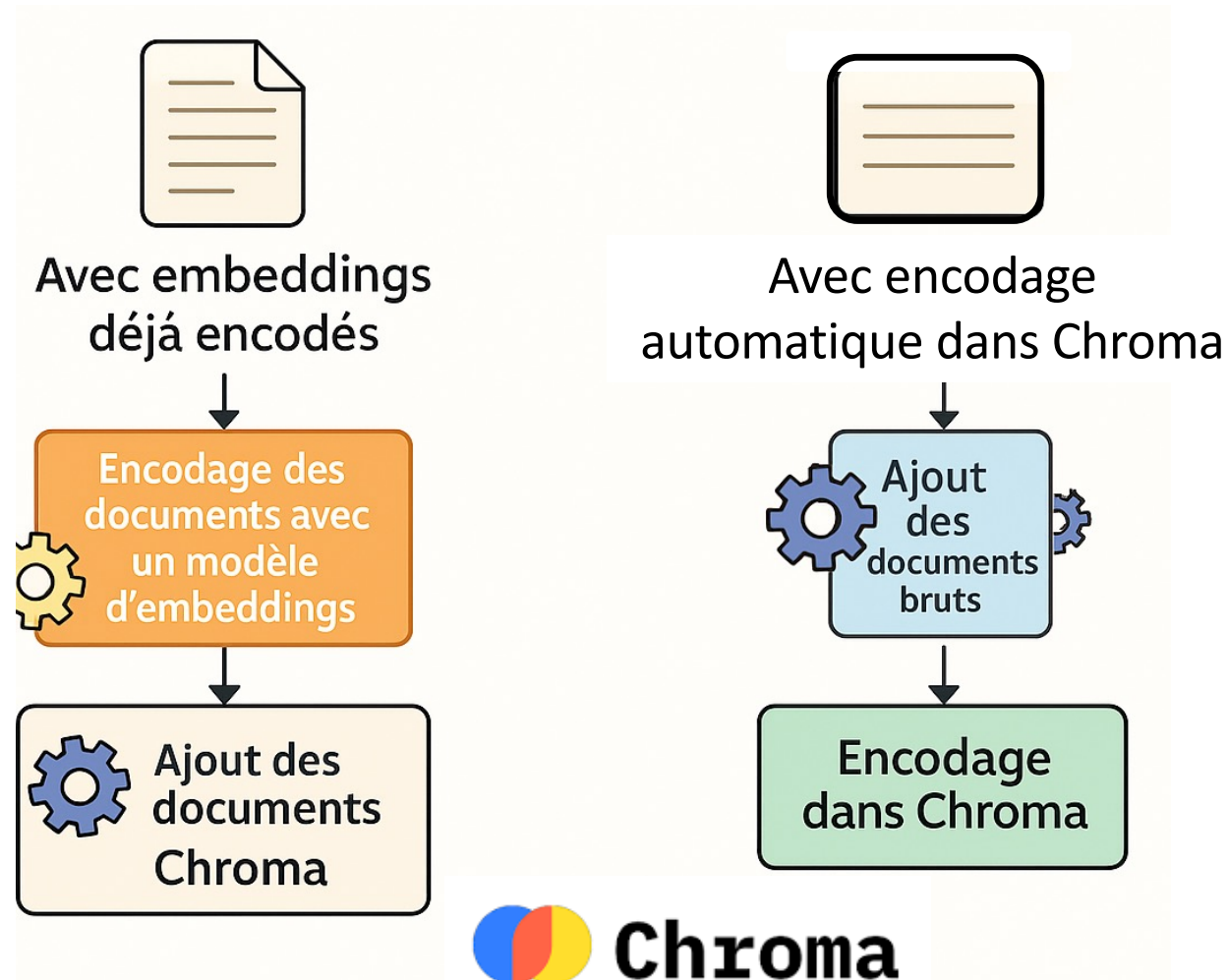
- ◆ Les quatre listes doivent être **de même longueur**.
- ◆ Les `ids` servent à relier documents ↔ embeddings ↔ métadonnées.

Un exemple simple de corpus

```
docs = [  
    "Le chat chasse une souris.",      # animaux  
    "Le chien aboie fort.",            # animaux  
    "Le poisson nage dans l'aquarium.", # animaux  
    "Le cuisinier prépare un gâteau.",  # cuisine  
    "La soupe mijote dans la casserole." # cuisine  
]  
  
metas = [  
    {"cat": "animaux"},  
    {"cat": "animaux"},  
    {"cat": "animaux"},  
    {"cat": "cuisine"},  
    {"cat": "cuisine"}  
]  
  
ids = ["doc1", "doc2", "doc3", "doc4", "doc5"]
```

Les 3 listes ont la même dimension

Création d'une collection



Création d'une collection avec embeddings encodés

```
# choix d'un modèle d'embedding
```

```
MODEL_NAME = "all-MiniLM-L6-v2"
```

```
model = SentenceTransformer(MODEL_NAME)
```

```
# encodage des documents
```

```
E = model.encode(docs, normalize_embeddings=True)
```

```
# creation de la collection
```

```
collection = client.get_or_create_collection(name="demo",  
      metadata={"hnsw:space": "ip"})
```

```
# ajout des documents
```

```
collection.add(  
    documents=docs,  
    ids=ids,  
    metadatas=metas,  
    embeddings=E.tolist() )
```

Choix IP (Produit scalaire) distance=produit scalaire
: similarité cosinus si les vecteurs sont normalisés

On fournit les embeddings déjà encodés

```
# vecteurs précédemment encodés
```

Collection avec encodage dans chroma

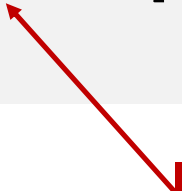
```
from chromadb.utils import embedding_functions
# choix d'une fonction d'embedding
embedder =
embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)
# creation de la collection
client = Client()
collection = client.get_or_create_collection(name="demo_auto",
    embedding_function=embedder,
    metadata={"hnsw:space": "ip"} )
# ajout des documents
collection.add(
    documents=docs,
    ids=ids,
    metadatas=metas)
```

Chroma encode
automatiquement les vecteurs

Requêtes d'ajout

- Ajout d'un document dans une collection

```
collection.add(  
    documents=["le chat dort"],  
    metadatas=[{"categorie": "animaux"}],  
    ids=["doc0"]  
)
```



Attention :L'ids doit être unique

Requêtes de mise à jour

- Modification et suppression de documents dans une collection
- Accès par leur ids

Modification

```
collection.update(  
    ids=["doc1"],  
    documents=["Le chat chasse des souris."],  
)
```

Suppression

```
collection.delete(ids=["doc0"])
```

Deux types d'accès aux documents

GET : Accès direct

- lecture exacte du contenu d'une collection
- Accès par ids ou filtrage sur les métadonnées (where)

```
res = collection.get(  
    ids=["doc1"],  
    # Identifiants précis  
    include=["documents", "metadatas"]  
)
```

Resultat dictionnaire de listes

QUERY : recherche par similarité

- Recherche par **similarité vectorielle** (embeddings)
- Basé sur une requête textuelle ou vectorielle
- Peut être combiné à un filtre symbolique (where)

```
res = collection.query(  
    query_texts=["chat"],  
    n_results=3,  
    include=["documents", "metadatas"]  
)
```

Resultat dictionnaire de listes de listes

GET : accès direct par ids

- Accès direct à des documents connus à partir de leurs ids

```
res = collection.get(  
    ids=["doc1"],    # Identifiants précis  
    include=["documents", "metadata"]  
                    # Choix des éléments à récupérer  
)
```



```
{  
  "ids": ["doc1"],  
  "documents": ["Le chat chasse une souris."],  
  "metadata": [{"categorie": "animaux"}]  
}
```

GET : accès direct avec filtre where sur les métadonnées

- Accès direct à tous des documents avec cat=animaux

```
res = collection.get(  
    include=["documents"],  
    where={"cat": "animaux"}  
)  
print("Documents trouvés :")  
for doc in res["documents"]:  
    print(f"-{doc}")
```

Documents trouvés :

- Le chat chasse des souris.
- Le chien aboie fort.
- Le poisson nage dans l'aquarium.

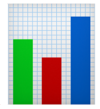
QUERY : Requête vectorielle avec query_texts

👉 requête top-k = k=n_results

```
res = collection.query(  
    query_texts=["chat chasse"],  
    n_results=3,  
    include=["documents", "metadatas", "distances"]  
)
```

id	cat	similarité	document
doc1	animaux	0.368	Le chat chasse une souris.
doc5	cuisine	0.744	La soupe mijote dans la casserole.
doc2	animaux	0.757	Le chien aboie fort.

💡 La distance correspond au produit scalaire c'est donc la similarité (si les embeddings normalisés).



Comprendre la structure du résultat (res)

```
print(res.keys())  
# → dict_keys(['ids', 'distances', 'documents',  
               'metadata'])
```

Chaque clé contient **une liste de listes** (listes simples pour GET!!):

```
res["ids"][0]           # ['doc_1', 'doc_0', 'doc_2']  
res["documents"][0]     # textes correspondants  
res["metadata"][0]      # [{...}, {...}, {...}]  
res["distances"][0]     # [0.08, 0.62, 0.74]
```



`res["ids"][i]` = résultats pour la i^{e} requête du batch



Query avec Filtres sur métadonnées

Requête hybride

1 Filtre simple

```
res = col.query(  
    query_texts=["chat chasse"],  
    n_results=3,  
    where={"cat": "animaux"}  
)
```

La recherche vectorielle est exécutée après avoir filtré les documents par le filtre where

doc1	animaux	0.368	Le chat chasse une souris.
doc2	animaux	0.757	Le chien aboie fort.
doc3	animaux	0.881	Le poisson nage dans l'aquarium.

2 Combinaisons logiques

ET

```
where={"cat": "animaux", "source": "blog"}
```

OU

```
where={"$or": [{"cat": "animaux"}, {"cat": "météo"}]}
```

Requête vectorielle pure

```
q_vec = model.encode(["chat sur canapé"],  
                      normalize_embeddings=True)
```

modèle d'embeddings ex: Sbert

```
res = col.query(  
    query_embeddings=q_vec,  
    # pas de query_texts  
    n_results=2,  
    include=["documents", "metadatas", "distances"]  
)
```

vecteur numpy ou liste



ATTENTION : le modèle doit être le même que celui qui a encodé le corpus

Chroma sous le capot

API Python Chroma

+ `collection.add(documents=docs)`

Q `res = collection.query(query_texts=["chat"],`

Index vectoriel FAISS



Indexs vectoriel FAISS



Stockage persistant



Chroma : une surcouche de gestion de corpus

Chroma sert d'interface haut-niveau :

- Elle gère **les collections**, c'est-à-dire des ensembles de documents + leurs embeddings + métadonnées
- Elle assure la **persistance** sur disque (via DuckDB)
- Et elle délègue la **recherche de similarité** à un moteur bas-niveau...
👉 en général **FAISS** lorsqu'on installe Chroma localement avec `pip install chromadb`

Un Exemple Complet

A tester dans le notebook du cours
TPCours-BDVectorielleChroma.pynb



1 - Initialiser le SGBD et créer une base (collection)



```
# pip install chromadb sentence-transformers
import chromadb
from chromadb config import Settings

# 1) Client SGBD (persistance locale)
client = chromadb Client(Settings(
    is_persistent=True,
    persist_directory="chroma_data"    # dossier de la base
))

# 2) Créer / récupérer une "collection" (la base vectorielle)
collection = client get_or_create_collection(
    name="articles_demo",
    metadata={"hnsw:space": "cosine"}    # espace de similarité (cosine|l2|ip)
)

print("Collections existantes :", client list_collections())
```



Collections existantes : [Collection(name=articles_demo)]

2 - Encodage : produire des embeddings



```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")
docs = [
    "La banque centrale relève ses taux directeurs ",
    "Le chat dort sur le canapé du salon ",
    "Il va pleuvoir demain "
]
ids = [f"doc_{i}" for i in range(len(docs))]
metas = [{"source": "actu", "cat": "économie"},
         {"source": "blog", "cat": "animaux"},
         {"source": "meteo", "cat": "météo"}]

# Embeddings normalisés (utile pour cosinus)
E = model.encode(docs, normalize_embeddings=True)
```

Utilisation d'un modèle
Sentence BERT

liste d'identifiants textuels IDS de la forme
"doc_0", "doc_1", "doc_2", etc
pour référencer les embeddings

métadonnées des
documents

3- Insertion d'embeddings et affichage Chroma

```
# Insertion dans la collection  
(SGBD)
```

```
collection.add(  
    documents=docs,  
    ids=ids,  
    metadatas=metas,  
    embeddings=E.tolist()  
)
```

```
# Récupérer les documents stockés
```

```
# get() retourne un dictionnaire  
avec ids, documents, embeddings,  
metadatas
```

```
docs = collection.get()
```

```
# Affichage des documents
```

```
print("Nb d'objets :",  
      collection.count())  
for doc_id, doc_text, meta in  
zip(docs["ids"], docs["documents"],  
    docs["metadatas"]):  
    print(f"ID: {doc_id}")  
    print(f"Texte: {doc_text}")  
    print(f"Métadonnées: {meta}")  
print("-" * 40)
```

```
Nb d'objets : 30  
ID: doc_000  
Texte: La banque centrale relève ses taux directeurs.  
Métadonnées: {'cat': 'économie'}  
-----  
ID: doc_001  
Texte: Le gouvernement présente un budget de rigueur.  
Métadonnées: {'cat': 'économie'}  
-----  
ID: doc_002  
Texte: L'inflation ralentit selon les derniers chiffres.  
Métadonnées: {'cat': 'économie'}  
-----
```

4- Requêtes Top-k



Requête vectorielle (embedding calculé explicitement)

```
q = "La banque centrale annonce"
```

```
q_emb = model.encode([q], normalize_embeddings=True) # encodage de la requête
```

```
res = collection.query(  
    query_embeddings=q_emb.tolist(),
```

→

0.258	économie	La banque centrale relève ses taux directeurs.
0.536	économie	La banque annonce des bénéfices record cette année.
0.741	politique	Le président visite la capitale pour un sommet.

```
n_results=3,
```

```
include=["documents", "metadatas", "distances"]
```

```
#contrôle ce qui est renvoyé (docs/ids/metas/embeddings/distances)
```

```
)
```

```
print("Requête :", q
```

```
for doc, meta, dist in zip(res["documents"][0], res["metadatas"][0],  
res["distances"][0]):
```

```
    print(f"{dist: 3f} | {meta['cat']:<8} | {doc}")
```

En espace ip, Chroma renvoie
directement s-des similarités
cosinus dans les distances

5- Filtrage, mise à jour, suppression

Filtrer par métadonnées

```
res = collection.query(  
    query_embeddings=q_emb.tolist(),  
    n_results=3,  
    where={"cat": "animaux"},  
    include=["documents", "metadatas", "distances"]  
)
```

Mise à jour (upsert) : réécrire un doc existant

```
collection.update(  
    ids=["doc_1"],  
    documents=["Le chat ronronne près du poêle "],  
    metadatas=[{"source": "blog", "cat": "animaux"}]  
)
```

Suppression

```
collection.delete(ids=["doc_2"])
```



Exemples de filtres

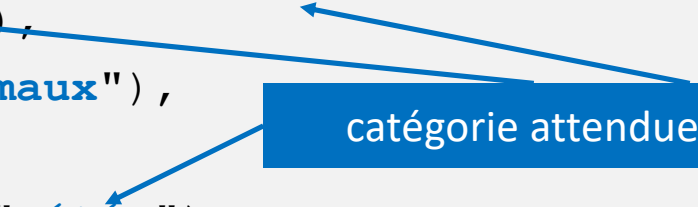
- Égalité : {"cat": "économie"}
- In : {"cat": {"\$in": ["météo", "animaux"]}}
- Numérique : {"score": {"\$gte": 0.8}}

6- Évaluation : calculs P@k / R@k sur une liste de requetes

Mini-évaluation P@k / R@k

```
eval_set = [  
    ("Hausse des taux d'intérêt", "économie"),  
    ("Prévisions d'inflation et budget de l'État", "économie"),  
    ("Un félin qui fait la sieste", "animaux"),  
    ("Un chien qui joue dans le jardin", "animaux"),  
    ("Risque de pluie demain", "météo"),  
    ("Alerte canicule la semaine prochaine", "météo"),  
    ("Trafic dense sur l'autoroute", "transport"),  
    ("Retard d'un train sur le quai", "transport"),  
    ("Le président s'exprime au sommet", "politique"),  
    ("Le parlement discute d'une réforme", "politique"),  
    ("Ouverture d'une exposition d'art", "culture"),  
    ("Un festival de musique commence vendredi", "culture")]
```

catégorie attendue (gold)



6- Évaluation : calculs P@k / R@k sur une liste de requetes

```
def precision_recall_at_k(results_cats, target_cat, k):  
    #Fonction d'évaluation  
    topk = results_cats[:k] # Catégories des k premiers résultats  
    # Nombre de Vrais positifs (bons résultats)  
    tp = sum(1 for c in topk if c == target_cat)  
    # Nombre total de documents pertinents  
    total_rel = metas.count({"cat": target_cat})  
    precision = tp / k  
    recall = tp / max(1, total_rel) # pour éviter la division  
    par zéro  
    return precision, recall
```

6- Évaluation (2/3)



Boucle d'évaluation

```
K = 3
scores = []
for q, gold_cat in eval_set:
    #Chaque requête est transformée en embedding (q_emb)
    q_emb = model.encode([q], convert_to_numpy=True, normalize_embeddings=True)
    #On interroge Chroma pour récupérer les K=3 voisins les plus proches
    res = collection.query(query_embeddings=q_emb.tolist(), n_results=K,
                           include=["metadatas"])
    #On extrait les catégories des résultats (cats)
    cats = [m["cat"] for m in res["metadatas"][0]]
    #On calcule précision & rappel pour cette requête
    p, r = precision_recall_at_k(cats, gold_cat, K)
    #On stocke les scores dans scores
    scores.append((q, gold_cat, p, r))
```

```
eval_set = [
    ("Hausse des taux d'intérêt", "économie"), ...
```

```
ex: pour " Hausse des taux d'intérêt "
cats=['économie', 'économie', 'économie']
```

6- Évaluation (3/3)



Affichage détaillé du résultat

```
print("Precision/Recall@3")
print(f"{'Query':<30} | {'Gold':<10} | {'P@3':>5} | {'R@3':>5}")
print("-" * 60)
for q, gold, p, r in scores:
    print(f"{'q:<30'} | {'gold:<10'} | {'p:6 2f'} | {'r:6 2f'}")
```

Query	Gold	P@3	R@3
Hausse des taux d'intérêt	économie	1.00	0.60
Prévisions d'inflation et budget de l...	économie	1.00	0.60
Un félin qui fait la sieste	animaux	0.33	0.20
Un chien qui joue dans le jardin	animaux	1.00	0.60

3 documents de la catégorie "économie" dans le top3= 3/3 =1

Recall= 3/5 car 5 documents "économie" en tout

Précision & Rappel

Précision (P@k) : parmi les k premiers résultats, quelle proportion est correcte ? 🙌 *Qualité*

Rappel (R@k) : parmi tous les résultats pertinents du corpus, quelle proportion a été retrouvée dans les k premiers ? 🙌 *Couverture des résultats pertinents*

6- Evaluation

Résultats complets

Precision/Recall@3				
Query	Gold		P@3	R@3
Hausse des taux d'intérêt	économie		1.00	0.60
Prévisions d'inflation et budget de l...	économie		1.00	0.60
Un félin qui fait la sieste	animaux		0.33	0.20
Un chien qui joue dans le jardin	animaux		1.00	0.60
Risque de pluie demain	météo		0.67	0.40
Alerte canicule la semaine prochaine	météo		0.67	0.40
Trafic dense sur l'autoroute	transport		1.00	0.60
Retard d'un train sur le quai	transport		1.00	0.60
Le président s'exprime au sommet	politique		0.67	0.40
Le parlement discute d'une réforme	politique		0.67	0.40
Ouverture d'une exposition d'art	culture		0.67	0.40
Un festival de musique commence vendredi	culture		0.67	0.40

A tester sur le
notebook du cours

Persistance et bonnes pratiques

- Persistance : Chroma écrit automatiquement dans `chroma_data/`
 - Redémarrage :
 - `client = chromadb Client(Settings(is_persistent=True, persist_directory="chroma_data"))`
 - puis `collection = client.get_collection("articles_demo")`
- Bonnes pratiques
 - Normaliser les embeddings
 - IDs uniques (sinon update)



A vous de jouer : TP4- BDVectorielleChroma.pynb

Récupérez le TP4_BDVectorielleChroma.pynb

Plan du notebook

- 1 Installation & imports
- 2 Chargement d'un petit corpus (textes courts)
- 3 Génération d'**embeddings** (Sentence-Transformers MiniLM)
- 4 **Chroma** : création de collection, insertion, requêtes de similarité
- 5 Requêtes Chroma (Symboliques et vectorielles)
- 6 Mini-évaluation (précision@k / recall@k sur requêtes annotées)
- 7 Visualisation 2D de l'espace vectoriel (PCA)





La suite ... Partie 2

Mise en œuvre avancée des systèmes RAG

Explorer des embeddings plus puissants et les intégrer dans des systèmes RAG complets

👉 **Embeddings avancés et encodage sémantique**

Utilisation des **modèles modernes (LLM)** pour générer des embeddings

👉 **Mise en œuvre de systèmes RAG complets**

Recherche + Génération (retrieval augmented generation)

Orchestration de pipelines IA

Supervision, biais, évaluation